

Bronze Layer - Data Engineering Reference

PostgreSQL + Python | CSV ingestion | Idempotency | Transactions | Lineage | Validation

Purpose: A compact reference for the Bronze layer you built, including architecture, responsibilities, transaction design, failure handling, and commands.

1. What the Bronze layer does

Bronze is the raw ingestion layer. It captures source data reliably with minimal transformation and attaches operational metadata so loaded rows can be traced to an ingestion attempt.

Bronze SHOULD	Bronze SHOULD NOT
Preserve source values	Standardize IDs
Use permissive source types such as TEXT	Clean whitespace/casing
Add source_file, batch_id, ingested_at	Normalize gender/categories
Record ingestion history	Apply business rules
Support safe retries	Perform CRM/ERP integration

2. Sources and Bronze tables

Source	CSV	Bronze table
CRM	cust_info.csv	bronze.crm_cust_info
CRM	prd_info.csv	bronze.crm_prd_info
CRM	sales_details.csv	bronze.crm_sales_details
ERP	CUST_AZ12.csv	bronze.erp_cust_az12
ERP	LOC_A101.csv	bronze.erp_loc_a101
ERP	PX_CAT_G1V2.csv	bronze.erp_px_cat_g1v2

3. Project responsibilities

ingestion_utils.py - reusable operations: calculate_file_hash(), already_ingested(), log_started(), log_failed().

Individual loader files - dataset-specific temp table, COPY, target table, row-count validation, and SUCCESS update.

run_bronze.py - orchestration only; calls the six loaders in sequence using fail-fast behavior.

SQL migrations - schemas/tables, UUID migration, ingestion_log, and database-level idempotency protection.

4. Standard ingestion flow

```
FILE EXISTS?
|
v
CALCULATE SHA-256 HASH
|
v
ALREADY SUCCESSFULLY INGESTED? -- yes --> SKIP
| no
v
GENERATE batch_id (UUID)
|
v
LOG STARTED - commit independently
|
v
```

```
COPY CSV -> TEMP -> INSERT BRONZE -> VALIDATE -> MARK SUCCESS
|
+-- success -> COMMIT
+-- failure -> ROLLBACK -> LOG FAILED -> re-raise error
```

5. Why the temporary table exists

The CSV contains only source columns, while Bronze contains source columns plus metadata. COPY is positional, so the clean pattern is:

```
CSV source columns -> temporary table -> INSERT ... SELECT + source_file + batch_id -> Bronze
```

The temp table uses **ON COMMIT DROP**, so PostgreSQL removes it automatically when the transaction ends.

6. Bulk loading with COPY

Use PostgreSQL COPY instead of row-by-row INSERT. Python streams chunks to PostgreSQL through COPY FROM STDIN.

```
with SOURCE_FILE.open("r", encoding="utf-8") as f: with cur.copy(""" COPY temp_table(...) FROM STDIN WITH
(FORMAT CSV, HEADER TRUE) """) as copy: while data := f.read(8192): copy.write(data)
```

Chunk boundaries do not need to match CSV rows; PostgreSQL's COPY parser handles record boundaries.

7. Ingestion metadata and lineage

Column	Meaning
source_file	Human-readable source filename
batch_id	UUID identifying one ingestion attempt
ingested_at	Timestamp attached to Bronze rows
file_hash	SHA-256 identity of exact file contents
status	STARTED, SUCCESS, or FAILED
rows_loaded	Number of successfully loaded Bronze rows
error_message	Failure details when an attempt fails

Important: Generate `batch_id` exactly once in the main loader and pass it into `load_bronze()`.

8. Transaction design

Transaction 1 - STARTED: Insert the ingestion_log STARTED row and commit it, preserving evidence that the attempt began.

Transaction 2 - Data + SUCCESS: COPY, Bronze INSERT, validation, and SUCCESS update happen together. If any part fails, all data changes in this transaction roll back.

Transaction 3 - failure only: Update the committed STARTED row to FAILED, save `completed_at` and `error_message`, then commit.

Critical rule: Bronze data and SUCCESS belong in the same transaction. Otherwise data could commit while the log fails.

9. Idempotency, race conditions, and database protection

Idempotency: rerunning the same logical ingestion should not create duplicate effects.

Application check: `already_ingested(source_system, file_hash)` avoids unnecessary reprocessing.

Race condition: two processes can both check before either commits SUCCESS, so both may initially think loading is safe.

Database safeguard: a partial unique index prevents more than one SUCCESS for the same `source_system` + `file_hash`.

```
CREATE UNIQUE INDEX ux_ingestion_log_success ON bronze.ingestion_log (source_system, file_hash) WHERE
status = 'SUCCESS';
```

FAILED attempts may share the same hash. If concurrent loads race, the process that cannot mark SUCCESS gets an error; because SUCCESS and the Bronze INSERT share a transaction, that process's Bronze rows roll back.

10. Row-count validation

Count temp rows after COPY, insert into Bronze, then count Bronze rows for the current `batch_id`.

```
if bronze_rows != temp_rows: raise RuntimeError( f"Row-count mismatch: temp={temp_rows},
bronze={bronze_rows}" )
```

Returning `bronze_rows` from `load_bronze()` is for reporting to the caller; PostgreSQL does not require the return value.

11. Error-handling pattern

```
try: bronze_rows = load_bronze(batch_id) except Exception as e: log_failed(batch_id, e) raise else:
print( f"SUCCESS: {SOURCE_FILE.name} " f"({bronze_rows} rows, batch_id={batch_id})" )
```

Bare `raise` re-raises the original exception and preserves its traceback. Do not silently swallow ingestion failures.

12. Bronze runner

run_bronze.py orchestrates the loaders and does not duplicate ingestion logic. Fail-fast is appropriate at this stage.

```
def run_bronze(): print("Starting Bronze ingestion...") load_crm_customers() load_crm_products()
load_crm_sales_details() load_erp_customers() load_erp_loc() load_erp_px() print("Bronze ingestion
completed successfully.")
```

If one loader fails, the runner stops. After fixing it, rerun: previously successful files skip and the failed file retries.

13. Commands to remember

Activate project environment:

```
source "/Users/tina/1-Data Engineer/data_warehouse_project/.venv/bin/activate"
```

Run one loader:

```
python -m src.ingestion.load_crm_products
```

Run the complete Bronze pipeline:

```
python -m src.ingestion.run_bronze
```

Run a SOL migration:

```
psql -h localhost -p 5432 -U postgres -d data_warehouse -f
sql/bronze/006_add_ingestion_idempotency_index.sql
```

Exit the Python interactive shell (>>>):

```
exit() # or Ctrl+D on macOS
```

14. Validation queries

Check all Bronze table counts:

```
SELECT 'crm_cust_info' AS table_name, COUNT(*) FROM bronze.crm_cust_info UNION ALL SELECT 'crm_prd_info',
COUNT(*) FROM bronze.crm_prd_info UNION ALL SELECT 'crm_sales_details', COUNT(*) FROM
bronze.crm_sales_details UNION ALL SELECT 'erp_cust_az12', COUNT(*) FROM bronze.erp_cust_az12 UNION ALL
SELECT 'erp_loc_a101', COUNT(*) FROM bronze.erp_loc_a101 UNION ALL SELECT 'erp_px_cat_g1v2', COUNT(*)
FROM bronze.erp_px_cat_g1v2;
```

Check ingestion history:

```
SELECT source_system, source_file, status, rows_loaded, started_at, completed_at, error_message FROM
bronze.ingestion_log ORDER BY started_at;
```

15. Bronze completion checklist

- Six CSV feeds have individual loaders.
- Bulk ingestion uses PostgreSQL COPY.
- Source fields are preserved with permissive Bronze types.
- source_file, batch_id, and ingested_at provide row lineage.
- ingestion_log tracks STARTED / SUCCESS / FAILED attempts.
- SHA-256 hashes support duplicate detection.
- Python skips files that already have a SUCCESS.
- A partial unique index protects the SUCCESS invariant.
- Bronze INSERT and SUCCESS update commit atomically.
- Failures roll back Bronze data and are recorded separately.
- Row counts are validated per batch.
- run_bronze.py provides one fail-fast pipeline entry point.

Next: Silver. Apply proper data types, trimming/standardization, ID and gender normalization, deduplication, data-quality rules, and CRM/ERP integration.